Machine Translated by Google

Hierarchical method and k-means method (1)

ÿ Clustering using dendrograms in hierarchical clustering analysis

ÿ Utilization of Python's scipy package (scientific and technological functional processing) ÿ Dendrogram for 7 observation cases with 2 attributes (x, y)

ÿ [3, 4], [1, 5], [7, 9], [5, 4], [6, 8], [2, 3], [4, 4]

```
from scipy.cluster.hierarchy import dendrogram, linkage import numpy
as np from matplotlib
import pyplot as plt

data_ex = np.array([[3, 4], [1, 5], [7, 9],[5, 4], [6, 8], [2, 3],
                    [4, 4]])
plt.figure(figsize=(12,6))
cluster=linkage(data_ex ,method='ward')

dendrogram(cluster)
plt.title('Dendrogram', size=20)
plt.xlabel('sample data index (0, 1, ...)', size=20)
plt.ylabel('euclidean distance', size=20)
```

Import module for dendrogram

single(shortest linkage)

- Use the shortest distance between the two clusters

complete (longest linkage method) - Uses the farthest distance between two clusters

average(mean linkage method)

- Use the average of all distances between data points in each cluster

Ward (Ward linkage method)

- The 'central points and sum of squared errors' that occur when two clusters are merged

Use increments

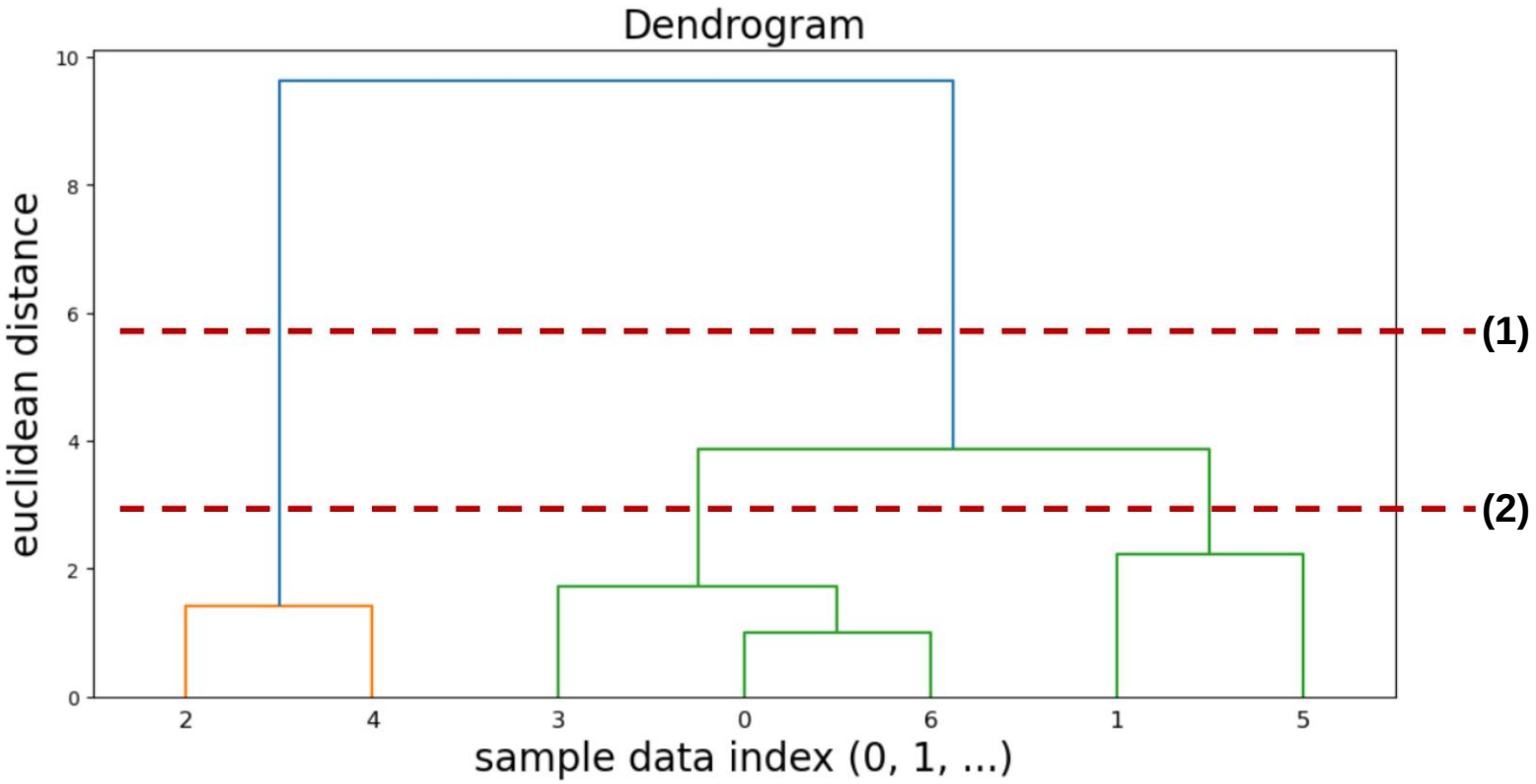
Machine Translated by Google

Hierarchical method and k-means method (2)

ÿ Clustering using dendrograms in hierarchical clustering analysis

ÿ From the dendrogram results below

ÿ The clustering results based on (1) are 2 clusters (2, 4) and (0, 1, 3, 5, 6). ÿ The clustering results based on (2) are 3 clusters (2, 4), (0, 3, 6), and (1, 5).



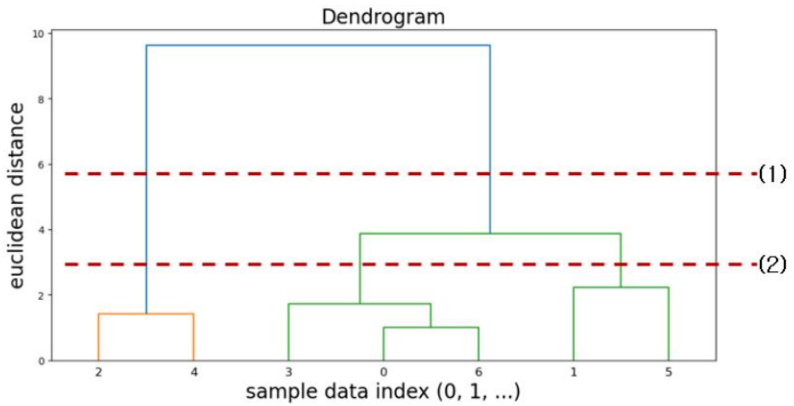
Machine Translated by Google

Hierarchical method and k-means method (3)

ÿ Clustering using dendrograms in hierarchical clustering analysis

ÿ In the dendrogram results below, 2

or 3 clusters can be visually identified through the lines (1) and (2), and cluster partitioning based on the number of clusters (k) is also possible through the program.




```
from sklearn.cluster import AgglomerativeClustering
import numpy as np
data_ex = np.array([[3, 4], [1, 5], [7, 9],[5, 4], [6,
                                    8], [2, 3],[4, 4]])
cluster = AgglomerativeClustering(n_clusters=2,
                                affinity='euclidean', linkage='ward')
cluster.fit_predict(data_ex)
```

```
array([0, 0, 1, 0, 1, 0, 0], dtype=int64)
```

```
from sklearn.cluster import AgglomerativeClustering
import numpy as np
data_ex = np.array([[3, 4], [1, 5], [7, 9],[5, 4], [6,
                                    8], [2, 3],[4, 4]])
cluster = AgglomerativeClustering(n_clusters=3,
                                affinity='euclidean', linkage='ward')
cluster.fit_predict(data_ex)
```

```
array([2, 0, 1, 2, 1, 0, 2], dtype=int64)
```

Machine Translated by Google

Hierarchical method and k-means method (4)

ÿ k-means clustering analysis using scikit-learn sklearn.cluster.Kmeans

ÿ Clustering 6 observation cases with 3 attributes (x, y, z) into 2 groups (k=2) ÿ [1, 2, 2], [1, 4, 5], [1, 0, 2], [10, 2, 7], [10, 4, 5], [10, 0, 6]

```
from sklearn.cluster import KMeans
import numpy as np

print(f'cluster: {kmeans.labels_}')
print(f'Center points: {kmeans.cluster_centers_}')
print(f'Cohesion: {kmeans.inertia_}')
kmeans.score(X)
```

- Import module to utilize k-means
- ÿ Set the model to use (K-means) and model name (kmeans) ÿ n_clusters: Set the number of clusters (2)
- k-means clustering operation
- Output k-means clustering results

```
cluster: [0 0 0 1 1 1]
Center: [[ 1. 2. 3.] [10. 2. 6.]]
Cohesion: 24.0
-24.0
```

ÿ Cohesion: The relationship between the centers of each cluster and the observations included in the cluster

The sum of the squares of the distances